Digital Mirror — Complete Technical Documentation

How the System Works, Why It Works, and Why It Can Be Trusted

Table of Contents

1. System Overview
2. Sensor Layer — The Hardware
3. Beat Detection — How Pulse is Found in Raw Data
4. BPM Calculation — Rolling Average
5. HRV Calculation — The Science of RMSSD
6. GSR Measurement and Drift Correction
7. Serial Output — What the ESP32 Sends and When
8. OLED Display — What the User Sees on the Device
9. Python Pipeline — How Data Flows Into the System
10. Calibration Phase — Building a Personal Baseline
11. 4-State Classification — The 2D Emotion Model
12. State Confirmation — Preventing False Readings
13. Sending to TouchDesigner — When and What
14. Accuracy Justification — Why This System Can Be Trusted
15. Known Limitations

1. System Overview

The Digital Mirror is a real-time biofeedback installation. It measures two involuntary physiological signals — **heart rate variability (HRV)** and **galvanic skin response (GSR)** — to infer the user's current emotional state. This state is then visualised as a fluid, colour-coded projection that surrounds the user's silhouette.

The pipeline has three stages:

```
[MAX30102 + GSR Sensor]
        ↓ (I2C + Analog)
[ESP32 Microcontroller]  ← Senses, filters, computes BPM + HRV
        ↓ (USB Serial, 115200 baud, every 500ms)
[Python osc.py]          ← Calibrates, analyses, classifies
        ↓ (OSC/UDP, port 7000)
[TouchDesigner]          ← Visualises the 4 emotional states
        ↓ (HDMI)
[Projector]
```

The key principle: **no single reading is ever trusted.** Every value is averaged, confirmed multiple times, and compared against the user's personal baseline before it changes what is shown on screen.

2. Sensor Layer — The Hardware

2.1 MAX30102 — Pulse and Heart Rate Sensor

The MAX30102 is a reflective photoplethysmography (PPG) sensor. It works by:

1. Shining an **infrared (IR) LED** into the user's fingertip.
2. A **photodetector** measures how much IR light bounces back from the blood.
3. When the heart beats and pumps blood into the finger, blood volume increases, absorbing more IR → the returned signal **dips**.
4. Between beats, blood volume decreases → IR signal **rises**.
5. This creates a rhythmic wave — each peak-to-trough cycle is one heartbeat.

The raw IR value is a number in the range of 0 to ~300,000 depending on the amount of reflected light. A typical resting finger gives readings of 70,000–150,000. A bare sensor (no finger) gives values under 30,000.

Settings used:

```
LED Amplitude:  0x1F  (31/255 — medium power, sufficient for finger contact)
Sample Rate:    400 Hz (400 samples per second read internally by the chip)
Pulse Width:    411 µs
ADC Range:      4096  (full 18-bit dynamic range)
```

At 400 Hz sampling, the MAX30102 fills its internal FIFO buffer very quickly. `getIR()` retrieves the latest sample from that buffer every time it is called in the main loop.

2.2 GSR Sensor — Galvanic Skin Response

The GSR sensor passes a tiny, safe electrical current between two electrodes placed on the user's fingers. It measures the **skin's electrical resistance**.

- **Dry skin** = high resistance = low conductance.
- **Sweaty skin** = low resistance = high conductance.

The sweat glands in the fingertips are innervated **exclusively** by the sympathetic nervous system (the "fight or flight" system). When the brain triggers stress or arousal, sweat gland activity increases — even before the person consciously feels anything. This makes GSR an extremely sensitive and fast early-warning signal.

The GSR sensor outputs an **analog voltage** on pin 35 of the ESP32. This is converted to a 12-bit digital value (0–4095) by the ESP32's ADC:

- `analogReadResolution(12)` → 12-bit precision (4096 levels)
- `analogSetAttenuation(ADC_11db)` → reads up to 3.3V full scale

Typical values:

- Resting / calm: 1400–2200 (varies hugely between individuals)
- Acute arousal: rapid change (delta) of ±50–200 from rolling average

3. Beat Detection — How Pulse is Found in Raw Data

The raw IR signal is not a clean sine wave. It has a large, slow drift caused by ambient light, slight finger movement, and temperature variations in the sensor. To isolate the heartbeat from this drift, we apply a **DC filter** (also called a high-pass filter). This removes the slow baseline and keeps only the fast-changing beat signal.

The filter equation (in code):

```
irFiltered = DC_ALPHA * irFiltered + (1.0 - DC_ALPHA) * irRaw;
long irForBeat = (irRaw - (long)irFiltered) + 120000;
```

Where:

- `irFiltered` is the running estimate of the DC (baseline) component.
- `DC_ALPHA = 0.85` is the filter coefficient:
  - High alpha (e.g. 0.95) → slower response, removes more DC but can miss fast beats.
  - Low alpha (e.g. 0.70) → faster, but less effective at removing drift.
  - **0.85 is a standard value used in clinical PPG processing.**
- `irRaw - irFiltered` removes the DC component, leaving only the pulsing AC signal.
- `+ 120000` is an offset to keep the signal positive (required by `checkForBeat()`).

`checkForBeat()` is a library function from SparkFun's MAX30105 library. It uses a threshold-based peak detection algorithm: when the signal crosses a rising threshold, a beat is flagged. It also implements refractory periods (ignoring beats within 333ms of the last) to prevent double-counting.

4. BPM Calculation — Rolling Average

When a beat is detected, the time since the last beat is measured:

```
long delta = millis() - lastBeat;  // milliseconds between beats
```

BPM is derived from the beat-to-beat interval:

$$BPM = 60{,}000 \text{ ms/min} \div \text{delta ms/beat}$$

For example: if delta = 857ms → BPM = 60000 / 857 = 70 BPM.

To prevent noise from one bad beat reading corrupting the output, BPM is **averaged over the last 5 beats** using a circular buffer:

```
#define BPM_AVERAGE_SIZE 5

bpmRates[rateSpot++] = (byte)beatsPerMinute;
rateSpot %= BPM_AVERAGE_SIZE;

// When enough beats collected:
beatAvg = 0;
for (byte x = 0; x < BPM_AVERAGE_SIZE; x++)
    beatAvg += bpmRates[x];
beatAvg /= BPM_AVERAGE_SIZE;
```

Why 5 beats?

- At 60 BPM, 5 beats = ~5 seconds of data.
- At 80 BPM, 5 beats = ~3.75 seconds.
- 5 beats is the minimum window that provides meaningful averaging without being so wide that it can't respond to a genuine increase in heart rate over ~30 seconds.
- The previous version used 3 beats — too few. A single noisy reading at 90 BPM would shift a true average of 72 BPM to 78 BPM, triggering false stress flags.

Validity guard: BPM is only reported after `MIN_VALID_BEATS = 5` have been collected. Before that, `reportBPM = 0` is sent over serial. This prevents Python from trying to classify stress on a BPM calculated from just 2 beats.

Beat interval bounds:

```
MIN_BEAT_DELTA_MS = 333   // = 180 BPM maximum
MAX_BEAT_DELTA_MS = 1500  // = 40 BPM minimum
```

Any beat interval outside these bounds is rejected as noise (movement artifact, electrical interference, etc.). A healthy adult's heart rate is always between 40 and 180 BPM, so any delta outside 333–1500ms is physiologically impossible and is discarded.

5. HRV Calculation — The Science of RMSSD

5.1 What is HRV?

Your heart does not beat like a metronome. Even at a steady 60 BPM, the gap between individual beats varies — one interval might be 980ms, the next 1020ms, then 950ms. This variation is controlled by the **Autonomic Nervous System (ANS)**:

- **Parasympathetic nervous system (vagus nerve)** → slows heart, increases variation → **High HRV**
- **Sympathetic nervous system ("fight or flight")** → accelerates and rigidifies heart → **Low HRV**

High HRV = autonomic flexibility, adaptability, calm.
Low HRV = sympathetic dominance, stress, anxiety, fatigue.

This is a well-established clinical biomarker used in cardiology, sports science, and mental health research. Over 30,000 peer-reviewed papers have been published on HRV since the 1990s.

5.2 RMSSD — The Formula

RMSSD (Root Mean Square of Successive Differences) is the most commonly used short-term HRV metric. It specifically captures **high-frequency** heart rate fluctuations driven by the vagus nerve (parasympathetic branch).

Formula:

$$RMSSD = \sqrt{\left[(1 / (N-1)) \times \Sigma \left(RR(i+1) - RR(i) \right)^2 \right]}$$

Where $RR(i)$ is the i-th R-R interval, and N is the number of intervals.

Step-by-step example calculation:
R-R intervals collected (ms): [820, 860, 790, 900, 840, 810, 870, 830]

1. Compute successive differences: +40, –70, +110, –60, –30, +60, –40
2. Square each difference: 1600, 4900, 12100, 3600, 900, 3600, 1600
3. Mean of squared differences: 28,300 / 7 = 4,042.9

4. Take the square root: $RMSSD = \sqrt{4042.9} = 63.6\ ms$

Interpretation of RMSSD values:

RMSSD Range	Meaning
< 20 ms	Very high stress / poor autonomic recovery
20 – 50 ms	Normal range for most adults
50 – 100 ms	Good vagal tone — calm, resilient
> 100 ms	Athletic / deeply relaxed state

5.3 Implementation in ESP32 Code

The ESP32 stores R-R intervals in a **circular buffer** of size 8:

```
#define RR_WINDOW      8    // Store last 8 R-R intervals
#define MIN_RR_FOR_HRV 4    // Need at least 4 intervals to compute

long  rrBuffer[RR_WINDOW];  // Circular buffer
int   rrHead  = 0;          // Next write position
int   rrCount = 0;          // Number of valid entries
```

When a valid beat is detected, `addRRInterval(delta)` is called:

```
void addRRInterval(long deltaMs) {
  rrBuffer[rrHead] = deltaMs;
  rrHead = (rrHead + 1) % RR_WINDOW;
  if (rrCount < RR_WINDOW) rrCount++;

  if (rrCount < MIN_RR_FOR_HRV) {
    hrvRMSSD = 0;
    return;
  }

  long ordered[RR_WINDOW];
  int start = (rrHead - rrCount + RR_WINDOW) % RR_WINDOW;
  for (int i = 0; i < rrCount; i++)
    ordered[i] = rrBuffer[(start + i) % RR_WINDOW];

  float sumSqDiff = 0.0f;
  int   pairs     = rrCount - 1;
  for (int i = 0; i < pairs; i++) {
    float diff = (float)(ordered[i + 1] - ordered[i]);
    sumSqDiff += diff * diff;
  }
  hrvRMSSD = (int)sqrt(sumSqDiff / pairs);
}
```

Why a window of 8 intervals? It balances responsiveness and stability. 8 intervals equal ~7 seconds of data, which exceeds the clinical recommendation of a minimum of 5 successive intervals for short-term RMSSD while staying dynamic.

6. GSR Measurement and Drift Correction

6.1 On the ESP32 — Multi-sample Averaging

```
#define GSR_SAMPLES 10

int readGSR() {
  long sum = 0;
  for (int i = 0; i < GSR_SAMPLES; i++) {
    sum += analogRead(GSR_PIN);
    delayMicroseconds(100);
  }
  return (int)(sum / GSR_SAMPLES);
}
```

Each GSR reading is the average of 10 consecutive ADC samples with 100µs gaps to reduce the noise issues of the ESP32 ADC.

6.2 In Python — GSR Drift Correction (Rolling Delta)

To isolate genuine emotional responses from natural skin warming and electrode acclimatisation, the system tracks the *change* (delta) from a rolling average instead of absolute values:

```
GSR_WINDOW_SIZE = 30  # Last 30 readings

gsr_window.append(gsr)
gsr_rolling_avg = sum(gsr_window) / len(gsr_window)
gsr_delta = gsr - gsr_rolling_avg
```

- `gsr_delta > 0` → Conductance is rising (arousal increasing)
- `gsr_delta ≈ 0` → Conductance is stable
- `gsr_delta < 0` → Conductance is falling (calming down)

7. Serial Output — What the ESP32 Sends and When

Every 500ms, the ESP32 prints one line to serial:

```
IR=xxxxx BPM=xx HRV=xx GSR=xxxx Beats=x
```

Sampling at 2Hz (500ms) matches the physiological speed of the signals (GSR takes 2-5 seconds to develop) and avoids flooding Python's serial buffer.

8. OLED Display — What the User Sees on the Device

The OLED updates every 250ms to provide rapid visual feedback to the device operator. It displays the finger connection status, raw parameters, and computed averages (BPM and HRV) or loading states if data collection is in progress.

9. Python Pipeline — How Data Flows Into the System

The Python script `osc.py` receives serial data and runs it through 6 sequential stages:

1. **Stage 1: Validity Check** — Ensures IR > 50,000 to verify finger presence.
2. **Stage 2: Calibration** — Collects baseline metrics during the first 15 seconds.
3. **Stage 3: GSR Drift Correction** — Computes the rolling delta.
4. **Stage 4: 4-State Classification** — Maps HRV and GSR against baseline.

5. **Stage 5: State Confirmation** — Requires 3 consecutive agreements and an 8s minimum hold.
6. **Stage 6: Send to TouchDesigner** — Dispatches OSC messages via UDP.

10. Calibration Phase — Building a Personal Baseline

Because resting HRV and GSR vary drastically between individuals due to age, fitness, and baseline physiology, a fixed global threshold is ineffective. The system measures the user's personal resting baseline for 15 seconds after initial contact. This establishes a personalized 50% midpoint to evaluate subsequent emotional deviations.

11. 4-State Classification — The 2D Emotion Model

Based on Russell's Circumplex Model of Affect (1980), human emotions are mapped on a 2D plane of Valence (X-axis, derived from HRV) and Arousal (Y-axis, derived from GSR delta).

State	HRV Threshold	GSR Delta	Color Accent	Visual Style
ZEN	High (Above baseline +10ms)	Low (< +30)	Electric Violet	Slow, smooth, cool fluid
STRESS	Low (Below baseline -10ms)	High (≥ +30)	Hot Pink	Fast, erratic, fragmenting
FLOW	High (Above baseline +10ms)	High (≥ +30)	Cyan	Fast but structured waves
BURNOUT	Low (Below baseline -10ms)	Low (< +30)	Dim Teal	Slow, dim, barely moving

12. State Confirmation — Preventing False Readings

To avoid jarring color flickering caused by noise spikes, a **Two-Gate Confirmation System** is implemented:

- **Gate 1 (Consecutive Agreements):** A state must be classified 3 consecutive times (~1.5 seconds) to become a candidate.
- **Gate 2 (Minimum Hold Time):** A state cannot change unless at least 8 seconds have elapsed since the last committed shift.

13. Sending to TouchDesigner — When and What

State flags (e.g., `/state/zen`) are sent **only upon confirmed changes** to prevent redundant network spamming. Raw data values are transmitted separately every 1 second strictly for diagnostic overlays. TouchDesigner uses a 1.0s Filter CHOP to smooth transitions cleanly between states.

14. Accuracy Justification — Why This System Can Be Trusted

While the Digital Mirror is not a medical device or a lie detector, it functions accurately as a real-time autonomic nervous system monitor. By leveraging well-established physiological metrics (RMSSD for vagal tone and skin conductance for sympathetic activity) combined with numerous engineering safeguards (such as bounds checking, multi-sample averaging, and baseline calibration), the system avoids false positives and faithfully tracks the direction of emotional shifts.

15. Known Limitations

Limitation	Impact	Mitigation Strategy
Short-term HRV	Less statistically stable than a standard 5-minute clinical recording.	Tracks relative direction rather than clinical absolute values.
Movement Artifacts	Finger shifts distort PPG readings and introduce fake beats.	Interval filters (333–1500ms) reject physiological anomalies.
GSR Drift / Variation	Absolute resistance values vary wildly over time and between people.	Evaluates rolling delta windows instead of absolute value baselines.
Emotional Ambiguity	Fear and excitement generate identical skin conductance spikes.	Uses HRV valence mapping to split positive (FLOW) from negative (STRESS).